ANEXO 06

GUÍA DE DESPLIEGUE EN QA Y GUÍA DE PUESTA EN MARCHA LOCAL

Plataforma de Gestión de Información de Privacidad (PIMS) — GESTITEL E.I.R.L.

Documentación técnica de operación — Documento 06

Versión del sistema documentada: migraciones core/0001 – core/0043 · customer/0001 – customer/0004

Arequipa, Perú — Octubre 2026

TABLA DE CONTENIDO

A quién va dirigido 3
Qué contiene 3
Parte A — Guía de despliegue en el entorno QA 3
A.1 Arquitectura del entorno 3
A.2 Requisitos previos: DNS 4
A.3 Preparación del sistema 4
A.4 Usuario y directorio de servicio 4
A.5 PostgreSQL: rol y base de datos 4
A.6 Obtener el código fuente 5
A.7 Entorno virtual y dependencias 5
A.8 Aplicación web (frontend) 5
A.9 Archivo de configuración .env 5
A.10 Recolectar los archivos estáticos 6
A.11 Migraciones del esquema público 6
A.12 Tenant público y superadministrador 7
A.13 Alta de una empresa cliente 7
A.14 Servicio de Gunicorn (systemd) 8
A.15 Nginx 8
A.16 Cortafuegos 8
A.17 HTTPS con certificado por nombre de dominio 9
A.18 Verificación funcional 10
A.19 Procedimiento de actualización de una versión 10
A.20 Solución de problemas 11
Parte B — Guía de puesta en marcha local 12
B.1 Requisitos previos 12
B.2 Base de datos 12
B.3 Variables de entorno 12
B.4 Dependencias del backend 13
B.5 Migraciones 13
B.6 Tenant público, superadministrador y empresa de desarrollo 13
B.7 Datos de demostración (opcional) 14
B.8 Levantar el backend 14
B.9 Frontend en modo desarrollo 14
B.10 Pruebas automatizadas 15
B.11 Reiniciar la base de datos local 15
B.12 Errores comunes 15

A quién va dirigido

A personal técnico que deba **instalar, actualizar u operar** la plataforma Gestitel: administradores de servidores, equipos de infraestructura y desarrolladores que necesiten levantar el sistema en su propio equipo.

Qué contiene

1. **Parte A — Guía de despliegue en el entorno QA:** instalación completa en un servidor Ubuntu bajo el dominio `27k.pe`, procedimiento de actualización, activación de HTTPS y solución de problemas.
2. **Parte B — Guía de puesta en marcha local:** cómo levantar el backend y el frontend en un equipo de desarrollo con Windows.

Ambas comparten la misma arquitectura: un backend Django con multiempresa por esquemas de PostgreSQL (`django-tenants`) que sirve también la aplicación web (React) ya compilada.

> **Convención de marcadores.** Los valores entre `<...>` (por ejemplo `<IP_DEL_SERVIDOR>`, `<URL_DEL_REPOSITORIO>`, `<RAMA_DE_LIBERACION>`, `<CORREO_REAL>`) deben sustituirse por los datos reales del entorno. Ninguna contraseña ni clave real aparece en este documento: las que se muestran son marcadores que deben reemplazarse por valores robustos y propios de cada entorno.

Parte A — Guía de despliegue en el entorno QA

A.1 Arquitectura del entorno

Componente	Configuración
Sistema operativo	Ubuntu Server
Servidor web	Nginx (`server_name 27k.pe *.27k.pe`), sirve los archivos estáticos y actúa como proxy
Servidor de aplicación	Gunicorn administrado por `systemd`, 3 procesos, comunicación por socket Unix `/run/gunicorn-gestitel/gunicorn.sock`
Base de datos	PostgreSQL local, base `gestitel_db`, rol `gestitel`, escucha solo en `127.0.0.1`
Ruta del proyecto	`/opt/gestitel/gestitel`
Usuario de servicio	`gestitel` (sin privilegios de administrador)
Panel de plataforma	`https://27k.pe/admin/` (esquema `public`, superadministradores de Gestitel)
Empresa de demostración	`https://demo.27k.pe/` (empresa "Demo QA")
Archivos de configuración versionados	`deploy/nginx/gestitel.conf`, `deploy/gunicorn.conf.py`, `deploy/systemd/gunicorn-gestitel.service`, `deploy/env.qa.example`

Flujo de una petición: navegador → Nginx → Gunicorn → Django (resuelve la empresa por el subdominio, valida el token y los permisos) → PostgreSQL (esquema de esa empresa).

Convenciones: los comandos con `sudo` requieren privilegios de administrador; los que operan sobre el proyecto se ejecutan como el usuario de servicio (`sudo -u gestitel ...`). Salvo indicación contraria, todos los comandos se ejecutan desde `/opt/gestitel/gestitel`.

A.2 Requisitos previos: DNS

En el proveedor DNS del dominio deben existir estos registros apuntando a la IP del servidor:

Registro	Tipo	Valor
`27k.pe`	A	`<IP_DEL_SERVIDOR>`
`*.27k.pe`	A	`<IP_DEL_SERVIDOR>`

El registro comodín es imprescindible: cada empresa cliente vive en su propio subdominio (`demo.27k.pe`, `empresa.27k.pe`). El dominio raíz sirve el panel de plataforma (`/admin/`).

Verificar la propagación:

```
dig +short 27k.pe
dig +short demo.27k.pe
```

A.3 Preparación del sistema

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y python3-venv python3-pip postgresql postgresql-contrib nginx git
```

Versiones mínimas:

```
python3 --version    # 3.10 o superior
psql --version       # 14 o superior
nginx -v
```

> **No se instala Node.js en el servidor.** La aplicación web se compila fuera del servidor y se publica ya construida dentro del directorio `static/` del proyecto.

A.4 Usuario y directorio de servicio

```
# Usuario de sistema, dueño del proyecto.
sudo useradd --system --create-home --shell /bin/bash gestitel

# Directorio raíz del despliegue.
sudo mkdir -p /opt/gestitel
sudo chown gestitel:gestitel /opt/gestitel
```

A.5 PostgreSQL: rol y base de datos

```
sudo -u postgres psql <<'SQL'
CREATE ROLE gestitel WITH LOGIN PASSWORD '<CONTRASENA_ROBUSTA>';
CREATE DATABASE gestitel_db OWNER gestitel;
-- django-tenants crea y gestiona esquemas: el rol necesita poder crearlos.
ALTER ROLE gestitel CREATEDB;
SQL
```

Guardar la contraseña en el gestor de secretos: se usará en el archivo `.env` (paso A.9).

Confirmar que PostgreSQL **no** está expuesto a internet:

```
sudo -u postgres psql -c "SHOW listen_addresses;"   # debe devolver: localhost
```

Si no fuera `localhost`, ajustar `listen_addresses = 'localhost'` en `postgresql.conf` y reiniciar con `sudo systemctl restart postgresql`.

A.6 Obtener el código fuente

```
sudo -u gestitel git clone --branch <RAMA_DE_LIBERACION> <URL_DEL_REPOSITORIO> /opt/gestitel/gestitel
cd /opt/gestitel/gestitel
```

La URL puede ser HTTPS con un token de solo lectura o SSH con una llave de despliegue.

A.7 Entorno virtual y dependencias

```
sudo -u gestitel python3 -m venv /opt/gestitel/gestitel/venv
sudo -u gestitel /opt/gestitel/gestitel/venv/bin/pip install --upgrade pip
sudo -u gestitel /opt/gestitel/gestitel/venv/bin/pip install -r /opt/gestitel/gestitel/requirements.txt
```

`gunicorn` y `whitenoise` ya están incluidos en `requirements.txt`.

A.8 Aplicación web (frontend)

La aplicación web es una SPA construida con React 19, Vite 6, TypeScript y Tailwind CSS 4, en un repositorio propio (`frontend_gestitel`). Se compila **fuera del servidor**, en un equipo con Node.js 20 o superior:

```
cd frontend_gestitel
npm install
npm run build      # ejecuta la verificación de tipos (tsc) y luego vite build
```

La compilación usa la ruta base `/static/` y escribe el resultado directamente en el directorio `static/` del repositorio del backend (`../gestitel/static/`), vaciándolo antes. Ese contenido se publica junto con el código del backend, por lo que el servidor no necesita Node.js: Django entrega el `index.html` y Nginx sirve los recursos desde `/static/`. La aplicación se comunica con la API en el mismo dominio de la empresa (mismo origen), resolviendo la empresa por el subdominio.

Si el directorio `static/` no viajara con el código, puede sincronizarse desde el equipo de compilación:

```
rsync -avz --delete ./static/ gestitel@<IP_DEL_SERVIDOR>:/opt/gestitel/gestitel/static/
```

A.9 Archivo de configuración .env

```
cd /opt/gestitel/gestitel
sudo -u gestitel cp deploy/env.qa.example .env
```

Generar una clave secreta nueva para Django:

```
sudo -u gestitel venv/bin/python -c \
  "from django.core.management.utils import get_random_secret_key; print(get_random_secret_key())"
```

Editar el archivo (`sudo -u gestitel nano .env`) y ajustar como mínimo:

Variable	Valor
`SECRET_KEY`	El valor recién generado
`DB_PASSWORD`	La contraseña del rol `gestitel` (paso A.5)

Variable	Valor
`DEBUG`	`False`
`ALLOWED_HOSTS`	`.27k.pe,127.0.0.1,localhost`
`ROOT_DOMAIN`	`27k.pe`
`CSRF_TRUSTED_ORIGINS`	`https://27k.pe,https://*.27k.pe` (con `http://` mientras no se active HTTPS)
`SECURE_SSL_REDIRECT`, `SESSION_COOKIE_SECURE`, `CSRF_COOKIE_SECURE`	`True` con HTTPS activo (paso A.17); `False` solo si el entorno aún funciona por HTTP
`EMAIL_BACKEND`, `EMAIL_HOST`, `EMAIL_PORT`, `EMAIL_HOST_USER`, `EMAIL_HOST_PASSWORD`, `EMAIL_USE_TLS`, `DEFAULT_FROM_EMAIL`	Datos del servidor de correo saliente (notificaciones de incidentes). Sin ellos, los correos solo se imprimen en el registro y nada se interrumpe
`AI_SETTINGS_ENCRYPTION_KEY`	Clave maestra con la que se cifra la credencial del Asistente IA (ver abajo)

Asistente IA. El asistente de consultas teóricas no se configura por variables de entorno, salvo la clave maestra de cifrado. Generarla **una sola vez** en el servidor y guardarla en el gestor de secretos:

```
sudo -u gestitel venv/bin/python -c "from cryptography.fernet import Fernet;
print(Fernet.generate_key().decode())"
```

Sin esta clave el asistente permanece apagado (el sistema no se interrumpe). La credencial del proveedor del asistente y el modelo se cargan después desde el panel de plataforma (`/admin/` → **Configuración del Asistente IA**), con el botón **Probar conexión** y la casilla **Asistente activo**. La clave maestra debe ser propia de cada entorno; si se pierde o se cambia, hay que volver a cargar la credencial en el panel.

Proteger el archivo (nunca se versiona):

```
sudo chmod 600 /opt/gestitel/gestitel/.env
sudo chown gestitel:gestitel /opt/gestitel/gestitel/.env
```

A.10 Recolectar los archivos estáticos

```
sudo -u gestitel venv/bin/python manage.py collectstatic --noinput
```

Copia todo a `/opt/gestitel/gestitel/staticfiles/`, el directorio que sirve Nginx.

> WhiteNoise opera en **modo simple**, sin manifiesto ni nombres con huella, de forma intencional: la aplicación web referencia sus recursos con rutas fijas bajo `/static/`. No cambiar esta configuración sin volver a verificar la aplicación web.

A.11 Migraciones del esquema público

```
sudo -u gestitel venv/bin/python manage.py migrate_schemas --shared
```

Crea el esquema `public` (empresas, dominios, licencias, superadministradores, configuración de plataforma). Los esquemas de cada empresa se crean automáticamente al dar de alta la empresa (paso A.13), y en ese momento se siembra su configuración inicial: matrices de evaluación, rangos de tasación, controles del Anexo A y su Declaración de Aplicabilidad, cláusulas de ISO/IEC 27001, catálogos (categorías, amenazas, vulnerabilidades, tipos de control, etc.), el periodo inicial de evaluación de riesgos y las metas del Dashboard.

A.12 Tenant público y superadministrador

```
sudo -u gestitel venv/bin/python manage.py shell
```

```
from customer.models import Client, Domain
from django_tenants.utils import schema_context
from django.contrib.auth import get_user_model
User = get_user_model()

# Tenant público (su esquema ya lo creó migrate_schemas --shared).
public, _ = Client.objects.get_or_create(schema_name='public', defaults={'name': 'GESTITEL'})
Domain.objects.get_or_create(domain='27k.pe', tenant=public, defaults={'is_primary': True})

# Superadministrador de Gestitel (vive en el esquema public).
with schema_context('public'):
    if not User.objects.filter(username='superadmin').exists():
        User.objects.create_superuser('superadmin', '<CORREO_ADMIN>', '<CONTRASENA_ROBUSTA>')
exit()
```

La contraseña debe cumplir la política de complejidad de la plataforma.

A.13 Alta de una empresa cliente

La forma recomendada es el formulario integral del panel de plataforma: **`https://27k.pe/admin/`** → **Empresas** → **Crear empresa**, que en un solo paso crea la empresa, su subdominio, su licencia (30 días por defecto), el límite de documentos (opcional) y el usuario administrador inicial.

Como alternativa por consola (empresa de demostración `demo.27k.pe`):

```
from datetime import timedelta
from django.utils import timezone
from customer.models import Client, Domain, License
from django_tenants.utils import schema_context, tenant_context
from django.contrib.auth import get_user_model
from core.models.user_profile_model import UserProfile
User = get_user_model()

demo = Client(schema_name='demo', name='Demo QA')
demo.save()                     # crea el esquema y aplica sus migraciones
Domain.objects.create(domain='demo.27k.pe', tenant=demo, is_primary=True)

today = timezone.localdate()
with schema_context('public'):
    sa = User.objects.get(username='superadmin')
    License.objects.create(client=demo, start_date=today,
                           end_date=today + timedelta(days=30), status='A', created_by=sa)

with tenant_context(demo):
    u = User.objects.create_superuser('admin', '<CORREO_ADMIN_EMPRESA>', '<CONTRASENA_ROBUSTA>')
    UserProfile.objects.get_or_create(user=u, defaults={'must_change_password': True})
```

> Sin una licencia vigente, toda la API de la empresa responde `403 LICENSE_EXPIRED`. Con HTTPS por nombre (paso A.17), cada empresa nueva requiere además ampliar el certificado con su subdominio.

A.14 Servicio de Gunicorn (systemd)

```
sudo cp /opt/gestitel/gestitel/deploy/systemd/gunicorn-gestitel.service /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl enable --now gunicorn-gestitel
sudo systemctl status gunicorn-gestitel     # debe mostrar "active (running)"
ls -l /run/gunicorn-gestitel/gunicorn.sock  # el socket debe existir
journalctl -u gunicorn-gestitel -f          # registros en vivo
```

A.15 Nginx

El archivo versionado `deploy/nginx/gestitel.conf` **es la configuración HTTPS** del entorno: el servidor del puerto 80 redirige a HTTPS y el del puerto 443 usa el certificado de `/etc/letsencrypt/live/27k.pe/`. Por eso, en un servidor **nuevo**, el orden es:

1. Publicar primero una configuración solo HTTP (la versión anterior del archivo, disponible en el historial del repositorio: `git show 4b11878:deploy/nginx/gestitel.conf`), con las cookies `Secure` en `False` en el `.env`.
2. Emitir el certificado (paso A.17, puntos 1 y 2).
3. Copiar la configuración HTTPS versionada y activar las variables de HTTPS (paso A.17, puntos 3 y 4).

Si se copia la configuración HTTPS antes de tener el certificado, `nginx -t` falla porque no existen las rutas del certificado.

```
sudo cp /opt/gestitel/gestitel/deploy/nginx/gestitel.conf /etc/nginx/sites-available/gestitel
sudo ln -s /etc/nginx/sites-available/gestitel /etc/nginx/sites-enabled/gestitel
sudo rm -f /etc/nginx/sites-enabled/default   # evita que 27k.pe caiga en el sitio por defecto
sudo nginx -t
sudo systemctl reload nginx
```

Permisos obligatorios. Nginx corre como `www-data` y los archivos los crea `gestitel`. Sin estos permisos los estáticos responden 404 y la descarga de documentos (entregada de forma protegida mediante `X-Accel-Redirect`) responde 403:

```
# Recorrido hasta el proyecto.
sudo chmod o+x /opt /opt/gestitel /opt/gestitel/gestitel

# Acceso de grupo a los archivos subidos (requiere restart, no reload).
sudo usermod -aG gestitel www-data
sudo systemctl restart nginx
sudo groups www-data      # debe incluir "gestitel"
```

> Los archivos subidos **nunca se sirven de forma pública**: Nginx solo los entrega después de que la aplicación valida la sesión, el permiso del módulo y la empresa (ubicación interna `/protected_media/`).

A.16 Cortafuegos

```
sudo ufw allow OpenSSH
sudo ufw allow 'Nginx Full'     # puertos 80 y 443
sudo ufw enable
sudo ufw status
```

El puerto de PostgreSQL (5432) **no se abre**: la base de datos solo escucha en `127.0.0.1`.

A.17 HTTPS con certificado por nombre de dominio

El entorno QA usa certificados de Let's Encrypt emitidos **por nombre de dominio**, validados por HTTP (puerto 80) y con renovación automática.

1. Listar los nombres a cubrir (el dominio raíz y todos los subdominios de empresa):

```
sudo -u gestitel venv/bin/python manage.py shell -c "from customer.models import Domain; print('
'.join(Domain.objects.values_list('domain', flat=True)))"
```

2. Instalar certbot y emitir el certificado (primero una prueba en seco):

```
sudo apt install -y certbot python3-certbot-nginx
sudo certbot certonly --nginx --dry-run --non-interactive --agree-tos \
  -m <CORREO_REAL> -d 27k.pe -d demo.27k.pe -d <empresa>.27k.pe
# Si la prueba pasa, repetir sin --dry-run.
```

3. Activar HTTPS en Nginx copiando la configuración versionada, que ya es la configuración HTTPS:

```
sudo ufw allow 'Nginx Full'
sudo cp /opt/gestitel/gestitel/deploy/nginx/gestitel.conf /etc/nginx/sites-available/gestitel
sudo nginx -t && sudo systemctl reload nginx
```

4. Ajustar el `.env` y reiniciar la aplicación:

```
SECURE_SSL_REDIRECT=True
SESSION_COOKIE_SECURE=True
CSRF_COOKIE_SECURE=True
CSRF_TRUSTED_ORIGINS=https://27k.pe,https://*.27k.pe
```

```
sudo systemctl restart gunicorn-gestitel
```

5. Renovación automática:

```
systemctl list-timers | grep certbot
sudo certbot renew --dry-run
sudo certbot renew --deploy-hook "systemctl reload nginx"   # recargar Nginx tras renovar
```

6. Cada empresa nueva: registrar su dominio, verificar que resuelva al servidor y ampliar el certificado con todos los nombres:

```
sudo certbot certonly --nginx --expand --non-interactive --agree-tos -m <CORREO_REAL> \
  -d 27k.pe -d demo.27k.pe -d <empresa>.27k.pe -d <empresa-nueva>.27k.pe
sudo systemctl reload nginx
```

> Alternativa futura: un certificado comodín `*.27k.pe` mediante validación DNS-01, que evitaría ampliar el certificado por cada empresa. Requiere acceso automatizado al DNS del dominio.

A.18 Verificación funcional

```
# API de la empresa de demostración (resuelve la empresa por el Host):
curl -H "Host: demo.27k.pe" http://127.0.0.1/business/info/
# -> {"name": "Demo QA"}

# Panel de plataforma en el dominio raíz:
curl -I -H "Host: 27k.pe" http://127.0.0.1/admin/
# -> redirección al formulario de inicio de sesión

# Formulario público de derechos ARCO-P (no requiere inicio de sesión):
curl -H "Host: demo.27k.pe" http://127.0.0.1/business/arcop/info/
```

Desde un navegador:

- `https://27k.pe/admin/` → inicio de sesión del superadministrador.
- `https://demo.27k.pe/` → aplicación de la empresa; inicio de sesión con el administrador de la empresa.
- `https://demo.27k.pe/derechos-arcop` → formulario público ARCO-P. Enviar una solicitud crea un caso `ARCO-AAAA-####` visible en el módulo Solicitudes ARCO-P.
- Subir y descargar un documento en Gestión Documental → confirma que la entrega protegida de archivos funciona.

> El formulario público ARCO-P está **exento del control de licencia** (una empresa con licencia vencida sigue recibiendo solicitudes de los titulares, por obligación legal) y está limitado a 10 envíos por minuto por dirección IP.

Chequeo de configuración de Django:

```
sudo -u gestitel venv/bin/python manage.py check --deploy
```

A.19 Procedimiento de actualización de una versión

Cada vez que se publica una versión nueva en QA:

```
# 0) RESPALDO PRIMERO — obligatorio antes de cualquier migración.
sudo -u postgres pg_dump -F c gestitel_db -f ~/gestitel_db_$(date +%Y%m%d_%H%M).dump
ls -lh ~/gestitel_db_*.dump          # verificar que el archivo NO está vacío

cd /opt/gestitel/gestitel

# 1) Traer el código aprobado.
sudo -u gestitel git status          # confirmar que nadie editó archivos en el servidor
sudo -u gestitel git fetch origin
sudo -u gestitel git reset --hard origin/<RAMA_DE_LIBERACION>
sudo -u gestitel git log --oneline -3

# 2) Dependencias.
sudo -u gestitel venv/bin/pip install -r requirements.txt

# 3) Archivos estáticos.
sudo -u gestitel venv/bin/python manage.py collectstatic --noinput

# 4) Migraciones de TODOS los esquemas (public + cada empresa).
#    Leer la salida: las migraciones de datos informan por empresa qué sembraron o conservaron.
sudo -u gestitel venv/bin/python manage.py migrate_schemas

# 5) SOLO si la versión incorpora permisos nuevos (lo indican las notas de la versión).
sudo -u gestitel venv/bin/python mocks/populate_roles.py

# 6) Reiniciar la aplicación.
sudo systemctl restart gunicorn-gestitel
```

```
sudo systemctl status gunicorn-gestitel --no-pager
```

Después del reinicio, iniciar sesión en `https://demo.27k.pe/` y abrir algunos módulos. Si la pantalla se ve desactualizada, forzar la recarga sin caché del navegador.

Sobre la actualización de permisos (`populate_roles.py`):

- Es **aditiva e idempotente**: concede los permisos que falten sin borrar asignaciones hechas a mano. Ejecutarla de más no causa daño.
- Una sola ejecución recorre **todas** las empresas.
- Solo actualiza los **roles de sistema** (Administrador, Editor, Lector). Los roles personalizados de cada empresa deben recibir los permisos de un módulo nuevo desde **Usuarios y Roles**. Si tras una actualización un usuario "no ve" un módulo nuevo, revisar primero su rol.

Licencias tras una actualización. Los superadministradores nunca se bloquean por licencia, por lo que una prueba con su cuenta no revela una licencia vencida. Para revisar el estado real por empresa:

```
sudo -u gestitel venv/bin/python manage.py shell -c "
from customer.models import Client
from customer.services import evaluate_license_block, get_current_license
for c in Client.objects.exclude(schema_name='public'):
    lic = get_current_license(c)
    print(c.schema_name, evaluate_license_block(c) or 'OK', lic.end_date if lic else 'sin licencia')
"
```

Para renovar una licencia: panel de plataforma → **Licencias** → crear una fila nueva con fecha de fin posterior (el historial es inmutable). Una empresa **suspendida** devuelve `SERVICE_SUSPENDED`; una licencia **vencida**, `LICENSE_EXPIRED`.

Si algo sale mal: restaurar el respaldo del paso 0:

```
sudo -u postgres pg_restore --clean --if-exists -d gestitel_db ~/gestitel_db_AAAAMMDD_HHMM.dump
```

El respaldo es la única marcha atrás completa: algunas migraciones de datos son irreversibles en los datos (ver documento 04, inventario de migraciones).

A.20 Solución de problemas

Síntoma	Causa probable y solución
502 Bad Gateway	Gunicorn detenido o socket inaccesible. Revisar `systemctl status gunicorn-gestitel` y `journalctl -u gunicorn-gestitel -n 50`; tras una actualización suele ser una dependencia no instalada (paso 2).
Estáticos 404 (aplicación sin estilos)	Falta `collectstatic` o Nginx no puede leer `staticfiles/`. Repetir A.10 y los permisos de A.15.
404 de empresa en un subdominio	El dominio no está registrado. Verificarlo en el panel de plataforma.
`DisallowedHost` en los registros	El host no está en `ALLOWED_HOSTS`. Revisar `.env` y reiniciar Gunicorn.
403 LICENSE_EXPIRED en toda la API	La empresa no tiene licencia vigente. Crear o renovar la licencia.
403 CSRF en `/admin/`	Falta el origen en `CSRF_TRUSTED_ORIGINS` (con el esquema correcto, `https://`).
413 Request Entity Too Large al subir	Archivo mayor que `client_max_body_size` de Nginx o `MAX_UPLOAD_SIZE_MB`. Ajustar ambos de forma coherente.

Síntoma	Causa probable y solución
Descarga de documentos 403/404	Faltan permisos de grupo de `www-data` (A.15) o la ubicación interna no apunta al directorio real de archivos.
Advertencia de certificado en un subdominio nuevo	El certificado no incluye ese nombre. Ampliarlo (A.17, punto 6).
`nginx -t` falla tras copiar la configuración	Falta el certificado en `/etc/letsencrypt/live/27k.pe/`. Emitirlo primero (A.17) o usar temporalmente la configuración solo HTTP (A.15).
500 al arrancar	`SECRET_KEY` vacía o `.env` no cargado por systemd. Revisar `EnvironmentFile` y `journalctl`.

Parte B — Guía de puesta en marcha local

B.1 Requisitos previos

Requisito	Versión
Python	3.10 o superior
PostgreSQL	14 o superior (obligatorio: `django-tenants` no funciona con SQLite)
Node.js	20 o superior (solo para el frontend)
Repositorios	Backend (`gestitel`) y frontend (`frontend_gestitel`) en carpetas hermanas

B.2 Base de datos

Crear la base de datos (desde pgAdmin o una consola SQL):

```
CREATE DATABASE gestitel_db;
```

B.3 Variables de entorno

Copiar la plantilla versionada `.env.example` a `.env` en la raíz del backend y ajustar los valores. Contenido mínimo:

```
SECRET_KEY=<CLAVE_LARGA_PARA_DESARROLLO>
DEBUG=True
ALLOWED_HOSTS=127.0.0.1,localhost,.localhost
DB_NAME=gestitel_db
DB_USER=postgres
DB_PASSWORD=<CONTRASENA_DE_POSTGRES>
DB_HOST=127.0.0.1
DB_PORT=5432
```

Variables opcionales (todas tienen valor por defecto):

Variable	Por defecto	Uso
`ROOT_DOMAIN`	`localhost`	Dominio raíz del entorno (en QA: `27k.pe`)
`DEFAULT_LICENSE_DAYS`	`30`	Días de licencia al crear una empresa
`MAX_UPLOAD_SIZE_MB`	`10`	Tamaño máximo por archivo subido
`ALLOWED_UPLOAD_EXTENSIONS`	`doc,docx,xls,xlsx,ppt,pptx,pdf`	Extensiones permitidas al subir documentos
`EXCEL_IMPORT_MAX_ROWS`	`1000`	Máximo de filas en una importación por Excel

Variable	Por defecto	Uso
`API_PAGE_SIZE`	`20`	Tamaño de página de los listados (máximo 100)
`EMAIL_BACKEND`	Consola	En desarrollo los correos se imprimen en la consola; no requiere servidor de correo
`EMAIL_HOST`, `EMAIL_PORT`, `EMAIL_HOST_USER`, `EMAIL_HOST_PASSWORD`, `EMAIL_USE_TLS`, `DEFAULT_FROM_EMAIL`	—	Servidor de correo saliente (solo QA/producción)
`AI_SETTINGS_ENCRYPTION_KEY`	vacía	Clave maestra de cifrado del Asistente IA. Vacía, el asistente queda apagado y nada se interrumpe

B.4 Dependencias del backend

```
cd <RUTA_DEL_REPOSITORIO>\gestitel
python -m venv venv
.\venv\Scripts\activate
pip install -r requirements.txt
```

B.5 Migraciones

```
python manage.py migrate_schemas --shared   # crea el esquema public
```

Los esquemas de empresa se crean y migran al dar de alta cada empresa (paso B.6). Tras actualizar el código con migraciones nuevas, aplicar `python manage.py migrate_schemas` (todos los esquemas).

B.6 Tenant público, superadministrador y empresa de desarrollo

```
python manage.py shell
```

```
from datetime import timedelta
from django.utils import timezone
from customer.models import Client, Domain, License
from django_tenants.utils import schema_context
from django.contrib.auth import get_user_model
User = get_user_model()

# Tenant público
public, _ = Client.objects.get_or_create(schema_name='public', defaults={'name': 'GESTITEL'})
Domain.objects.get_or_create(domain='localhost', tenant=public, defaults={'is_primary': True})

# Superadministrador
with schema_context('public'):
    if not User.objects.filter(username='superadmin').exists():
        User.objects.create_superuser('superadmin', '<CORREO>', '<CONTRASENA>')

# Empresa de desarrollo (los datos de demostración esperan schema_name='negocio1')
neg = Client(schema_name='negocio1', name='Negocio Uno')
neg.save()      # crea el esquema, aplica sus migraciones y siembra la configuración inicial
Domain.objects.create(domain='negocio1.localhost', tenant=neg, is_primary=True)

# Licencia inicial (30 días)
today = timezone.localdate()
with schema_context('public'):
    sa = User.objects.get(username='superadmin')
    License.objects.create(client=neg, start_date=today,
                           end_date=today + timedelta(days=30), status='A', created_by=sa)
exit()
```

> En uso normal, la empresa, su dominio, su licencia y su administrador inicial se crean en un solo formulario: `http://localhost:8000/admin/` → **Empresas** → **Crear empresa**.

B.7 Datos de demostración (opcional)

```
python mocks\seeder.py
```

Puebla la empresa `negocio1` con roles y permisos, usuarios de demostración y datos de ejemplo de los módulos. Los usuarios de demostración y sus credenciales se definen en `mocks/json/usuarios.json`; los roles de sistema (Administrador, Editor, Lector) en `mocks/json/roles.json`. Tras incorporar permisos nuevos, `python mocks\populate_roles.py` actualiza los roles de sistema de todas las empresas sin borrar asignaciones manuales.

B.8 Levantar el backend

```
python manage.py runserver
```

- Aplicación de la empresa: `http://negocio1.localhost:8000` (el subdominio identifica a la empresa; los navegadores resuelven `*.localhost` sin modificar el archivo hosts).
- Panel de plataforma: `http://localhost:8000/admin/`. Los subdominios de empresa no exponen `/admin/`.
- Prueba rápida de la API:

```
curl http://negocio1.localhost:8000/business/info/
# -> {"name": "Negocio Uno"}
```

`http://localhost:8000` sin subdominio responde 404 de empresa: es el comportamiento esperado.

B.9 Frontend en modo desarrollo

Requisitos: Node.js 20 o superior y el backend en ejecución (paso B.8). La aplicación identifica a la empresa por el subdominio, por lo que se accede siempre con él.

```
cd <RUTA_DEL_REPOSITORIO>\frontend_gestitel
npm install
npm run dev          # servidor de desarrollo: http://negocio1.localhost:5173
```

Comando	Uso
`npm run dev`	Servidor de desarrollo con recarga automática
`npm run typecheck`	Verificación de tipos de TypeScript
`npm run lint`	Análisis estático (ESLint)
`npm run format`	Formato del código (Prettier)
`npm run test`	Pruebas automatizadas (Vitest, una pasada)
`npm run test:watch`	Pruebas en modo observación
`npm run build`	Verificación de tipos y compilación de producción hacia `../gestitel/static/`

En desarrollo, la aplicación apunta a la API en `http://<empresa>.localhost:8000`. Con `npm run build`, la aplicación compilada queda servida directamente por Django en `http://<empresa>.localhost:8000`, sin el servidor de desarrollo. El inicio de sesión siempre se valida contra la API real: sin backend en ejecución no es posible ingresar.

Organización del código (referencia):

Ruta	Contenido
`app/api/client.ts`	Cliente HTTP central: inyecta el token, lo renueva automáticamente ante un 401 y aplica los avisos de licencia y de permiso. Es el único punto que se comunica con el servidor
`app/services/`	Servicios por módulo, con la traducción entre el contrato de la API y el modelo de la interfaz
`app/components/common/`	Componentes reutilizables: tabla con paginación, filtros y orden en el servidor; tarjetas de indicadores; modales; barra de Excel; vista previa de PDF
`app/components/modules/`	Pantallas de cada módulo
`app/hooks/`	Lógica reutilizable de tablas, indicadores y permisos
`app/constants/modules.ts`	Registro único de módulos: alimenta el menú, las rutas y la matriz de permisos
`app/utils/saas.ts`	Resolución de la empresa por subdominio

B.10 Pruebas automatizadas

```
# Backend
python manage.py test core
python manage.py makemigrations --check     # confirma que no hay cambios de modelo sin migración

# Frontend (en frontend_gestitel)
npm run typecheck
npm run test
```

B.11 Reiniciar la base de datos local

```
DROP DATABASE gestitel_db;
CREATE DATABASE gestitel_db;
```

Después, repetir los pasos B.5 a B.7.

B.12 Errores comunes

Síntoma	Causa y solución
`fe_sendauth: no password supplied`	Falta el archivo `.env` o la variable `DB_PASSWORD`.
`relation "customer_client" does not exist`	Falta el paso B.5 (`migrate_schemas --shared`).
404 en `http://localhost:8000`	Correcto: el dominio raíz solo sirve `/admin/`. Usar `negocio1.localhost:8000`.
`403 LICENSE_EXPIRED` en toda la API	La empresa no tiene licencia vigente. Crearla o renovarla desde el panel.
404 en `negocio1.localhost:8000/admin/`	Correcto: el panel solo vive en el dominio raíz.

Síntoma	Causa y solución
Los datos de demostración fallan con `Client matching query does not exist`	La empresa debe llamarse `negocio1` (paso B.6).